The Perceptron to the MLP

Scaler School of Technology | Deep Learning I | Week 2 — Session 1/3 | Post-Lecture Review & Practice

Key Takeaways

1 A neuron is logistic regression

Weighted sum + bias + sigmoid = logistic regression. A single neuron draws one straight decision boundary. Simple but limited.

2 Activation functions add non-linearity

Without them, stacking layers does nothing (linear × linear = linear). ReLU is the default for hidden layers. Sigmoid for binary output. Vanishing gradients are why sigmoid/tanh struggle in deep networks.

3 MLPs learn complex boundaries through depth

One layer = lines. Two layers = shapes. Three layers = anything. The forward pass: multiply by weights, add bias, apply activation, repeat.

4 XOR proves depth matters

A single neuron can't solve XOR (not linearly separable). Adding one hidden layer with 2 neurons solves it. This is why "deep" learning is deep.

Test Your Understanding

Try answering these without looking at the slides. Answers are on page 3.

Q1. Explain in one sentence why stacking 100 linear layers without activation functions is equivalent to a single linear layer.

Q2. You have a binary classification problem. Which activation function do you use for the output layer? Which for hidden layers? Why?

Q3. Draw (or describe) how a 2-2-1 MLP solves XOR. What does each hidden neuron learn?

Q4. Why was Minsky & Papert's 1969 proof about the perceptron devastating for the field, even though the solution (MLPs) was conceptually simple?

Coding Exercises

Work through these in a Colab notebook. No starter code — figure out the approach from what you learned in the lecture.

Exercise 1: Neuron from Scratch

Implement a single neuron in PyTorch (no `nn.Module`). It takes 3 inputs, has 3 weights and a bias. Apply ReLU activation. Compute the output for input $[2.0, -1.0, 0.5]$ with weights $[0.5, -0.3, 0.8]$ and bias 0.1. Verify by hand.

Exercise 2: Activation Function Explorer

Plot sigmoid, tanh, ReLU, and LeakyReLU on the same graph for $x \in [-5, 5]$. Then plot their derivatives on a second graph. Which derivative goes to zero fastest for large $|x|$? Why does that matter for training?

Exercise 3: MLP Decision Boundaries

Use `sklearn.datasets.make_moons(n_samples=500, noise=0.2)`. Build 3 MLPs: (a) no hidden layer, (b) one hidden layer with 4 neurons, (c) two hidden layers with 16 and 8 neurons. Train each for 500 epochs. Plot all 3 decision boundaries side by side. How does depth affect the boundary shape?

Challenge Problem

The 3-Class Spiral

Generate `sklearn.datasets.make_blobs` with 3 classes arranged in a spiral pattern (or use a custom spiral dataset). Build an MLP that achieves $> 95\%$ accuracy. What's the minimum number of hidden layers and neurons needed? Plot the decision boundaries.

Further Reading

- **Goodfellow, Bengio & Courville — Deep Learning, Ch. 6** — deeplearningbook.org
  Deep Feedforward Networks — the definitive reference for MLPs and activation functions.
- **Michael Nielsen — Neural Networks and Deep Learning, Ch. 1** — free online at neuralnetworksanddeeplearning.org
  Excellent intuitive introduction to neural networks with interactive exercises.
- **3Blue1Brown — "But what is a neural network?"** — YouTube video
  Beautiful visual explanation of how neurons and layers work together.
- **Rumelhart, Hinton & Williams (1986) — Learning representations by back-propagating errors** — Nature paper
  The backpropagation paper that revived neural networks after the AI winter.

Online Resources & Tools

- **TensorFlow Playground** — interactive visualization of MLP decision boundaries
- **PyTorch nn.Module tutorial** — pytorch.org/tutorials for hands-on practice
- **Desmos graphing calculator** — plot activation functions live
- **CS231n notes on neural networks** — Stanford's deep learning course notes

Fill the Gaps — Prerequisite Refreshers

Felt lost on a specific concept? These resources will help you catch up.

- **Logistic regression** — Review scikit-learn's `LogisticRegression` docs and your Classical ML notes
- **Linear algebra (matrix multiply)** — 3Blue1Brown "Essence of Linear Algebra" Ch. 3
- **Python/NumPy** — NumPy quickstart tutorial at numpy.org
- **Gradient descent** — Preview: we cover this in Session 3

Answers — Test Your Understanding

A1. Matrix multiplication is associative: $W_{100} \times W_{99} \times \cdots \times W_1 =$ one single matrix W. No matter how many linear layers, the result is a single linear transformation. Activation functions break this chain.

A2. Output: sigmoid (squashes to 0–1 for probability). Hidden layers: ReLU (fast gradients, no vanishing gradient, computationally cheap). Sigmoid in hidden layers causes vanishing gradients in deep networks.

A3. Hidden neuron 1 learns one line that separates $(0, 0)$ and $(1, 1)$ from the others. Hidden neuron 2 learns another line. The output neuron combines these two signals with an AND/OR-like operation to produce the correct XOR output.

A4. The proof showed that a fundamental building block (the perceptron) couldn't solve a simple problem. Researchers concluded neural nets were a dead end. The solution (adding hidden layers + backpropagation) was conceptually simple but wasn't developed until 1986 — a 17-year gap. Most researchers had given up and moved to other approaches.

Hands-on notebook: www.priyanshsaxena.com/sst-deep-learning-nn **Instructor:** Priyansh Saxena